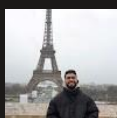




Set Up a Web App Using AWS and VS Code



Hayder Ben Abdelhamid

```
index.jsp X
src > main > webapp > index.jsp > html > body
1  <html>
2  <body>
3
4  <h2>Hello World It's Hayder</h2>
5
6  <p>This is my NextWork web application working!</p>
7  |
8  </body>
9  </html>
10
```



Introducing Today's Project!

This project is Day ONE of a seven day DevOps Challenge! In this project, i'm going to set up a foundations of a CI/CD pipeline by creating a web app from scratch. I need to launch a, EC2 instance and connect to the instance using VSCode to generate a web app inside.

This project is part one of a series of DevOps projects where I'm building a CI/CD pipeline! I'll be working on the next project soon

I did this project because its very rich and challenging project and it's great to set up a web app in the Cloud using EC2 and VS Code

Key tools and concepts

Services I used in this project were VS Code and Amazon EC2. Key concepts I learnt include SSH connections, using an IDE, launching an instance, editing an index.jsp file and using key pairs.

Project reflection

This project took me approximately 2 Hours. It was most rewarding to see a successful SSH connection to our EC2 instance, whether that's over the terminal or over the VS Code SSH connection.



Hayder Ben Abdelhamid

NextWork Student

nextwork.ai

One thing I didn't expect in this project was the secret mission where i also used the local terminal to try edit a file after using an IDE. It was fun to compare the difference in experience, and also still see the changes made over the terminal show up in in the vs Code window.

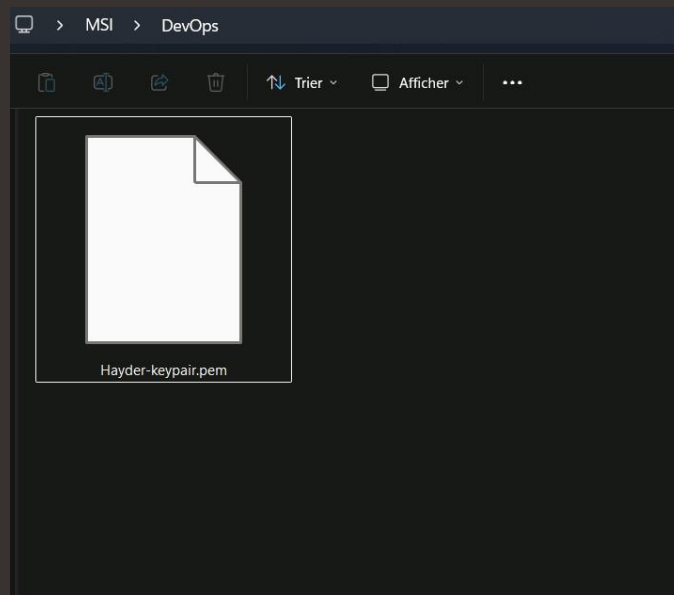


Launching an EC2 instance

What I did in this step

In this step, I will launching a EC2 instance in my AWS account! I'm also going to set up it's network settings (which helps me FIND my EC2 instance), and my key pair (which helps me make sure i have the PERMISSION to access the EC2 instance).

I started this project by launching an EC2 instance because EC2 instances are like virtual computers that live in the cloud! I want my web app to live entirely in the cloud, so i'm launching an EC2 instance to even develop my web app's code.





I also enabled SSH

SSH is the protocol that authorizes users (like ourselves) to access remote services (like EC2 instances). It's already a type of traffic that lets us transfer data back and forth with my EC2 instance once we're connected to it. I enabled SSH so that I can securely connect my EC2 instance later, and I made sure that it only allows SSH traffic from my IP address for security.

Key pairs

A key pair is a mechanism for us to connect and get access to EC2 instances I launch in AWS. I've created a key pair for the EC2 instance that I'm launching. Key pairs work by having two halves - a public key that AWS keeps, and a private key that I download. AWS will authenticate our private key when I want to connect to our EC2 instance later in this project, matching it to the private key that it has.

Downloaded key pair file

Once I set up my key pair, AWS automatically downloaded the private key file, called `nextwork-keypair.pem`. For safe keeping, I moved that private key into a folder called 'DevOps' in our Desktop.



Set up VS Code

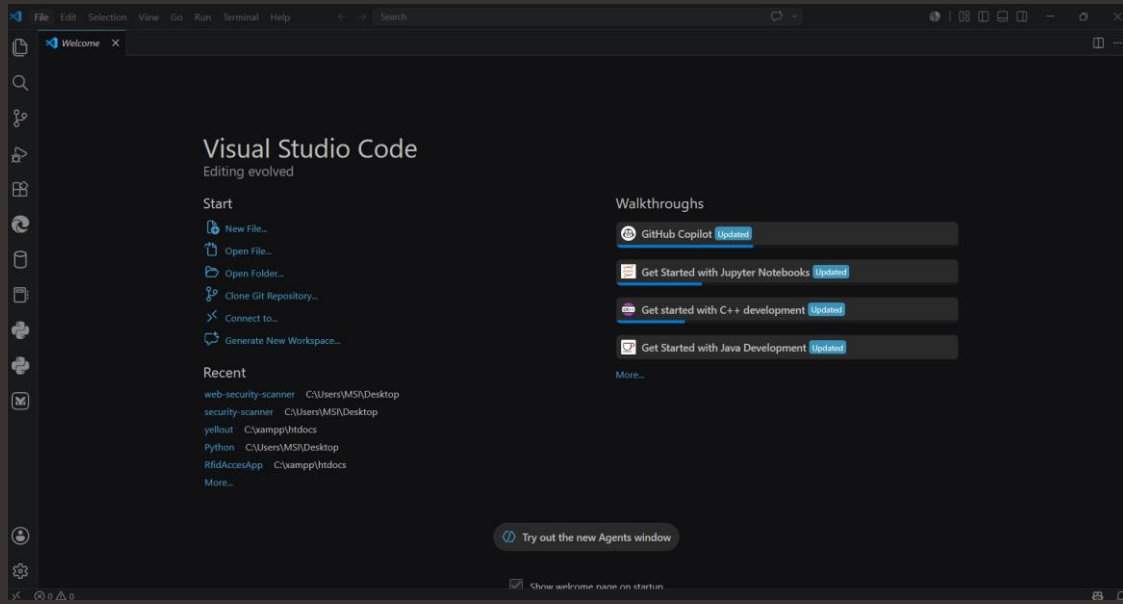
What I did in this step

In this step, I'm going to install VS Code, which is a tool that lets me write and edit code (and it's free). I'm also going to set up a terminal, and update the permission settings of my private Key.

What is VS Code?

VS Code is one of the most popular IDE's in the world; as an IDE, engineers use it to write code and manage their coding projects. It also has handy extensions that let us directly connect to an EC2 instance.

I installed VS Code to write and edit my web app's code. I'm also going to make the most out of ability to connect directly with an EC2 instance.





My first terminal commands

A terminal is a place for us to send commands to our computer i.e. tell it what we want it to do. The difference between a terminal and our usual click/drag interface is that we send TEXT commands in the terminal. The first command I ran for this project is `cd \Users\MSI\DevOps` which navigates the terminal to the DevOps folder.

Updating file permissions

I also updated my private key's permissions by running the commands `'icacls "Hayder-keypair.pem" /reset'`, `'icacls "Hayder-keypair.pem" /grant:r "MSI:R"` and the last one is `'icacls "Hayder-keypair.pem" /inheritance:r'`. This command gives me access to use this file so I can connect to a EC2 instance later.

```
C:\Users\MSI\DevOps>whoami
haider-msi\msi

C:\Users\MSI\DevOps>icacls "Hayder-keypair.pem" /reset
fichier traité : Hayder-keypair.pem
1 fichiers correctement traités ; échec du traitement de 0 fichiers

C:\Users\MSI\DevOps>icacls "Hayder-keypair.pem" /grant:r "MSI:R"
fichier traité : Hayder-keypair.pem
1 fichiers correctement traités ; échec du traitement de 0 fichiers

C:\Users\MSI\DevOps>icacls "Hayder-keypair.pem" /inheritance:r
fichier traité : Hayder-keypair.pem
1 fichiers correctement traités ; échec du traitement de 0 fichiers
```



SSH connection to EC2 instance

What I did in this step

Over the terminal, in this step, I'm going to try to connect to my EC2 instance! Once I've connected to the EC2 instance, I can start setting up my web app code inside.

Connecting to EC2

To connect to my EC2 instance, I ran the command `'ssh -i [path to private key] ec2-user@[ipv4 address of EC2 instance]'`. This command sets up an SSH connection directly between my local computer and the instance.

This command required an IPv4 address

A server's IPv4 DNS is like its public address that identifies when the server lives in the cloud. In our case, an EC2 instance's IPv4 DNS is useful information to give to our local computer - it tells our local computer where to find the EC2 instance that I launched.



Hayder Ben Abdelhamid

NextWork Student

nextwork.ai

```
      #_
     ~\ #####_      Amazon Linux 2023
    ~ ~\ #####\
    ~ ~ \###|
    ~ ~  \#/____
    ~ ~   V~' '->      https://aws.amazon.com/linux/amazon-linux-2023
    ~ ~
    ~ ~. _
    ~ ~  \_/_
    ~ ~   /m/'

[ec2-user@ip-172-31-18-195 ~]$
```



Maven & Java

What I did in this step

In this step, I will to install Maven and Java (corrected 8) in our EC2 instance! I'm doing this because both tools are helpful for setting up a Java web app from scratch.

Why I'm using Maven

Apache Maven is a tool that helps us with creating and organizing Java projects (like this web app). It comes with a lot of use cases, like being a package manager (downloading external pieces of code) and the tool that uses archetypes (templates) for spinning up projects like web apps instantly.

Maven is required in this project because I want to use its ability to spin web apps using archetypes! I'm about to set up my Java web app using the web app archetype.

Why I'm using Java

Java is programming language that I'm using to develop my web app. It's a very popular and versatile choice, as I can use it to develop all different kinds of applications, including web apps and enterprise systems.



Hayder Ben Abdelhamid

NextWork Student

nextwork.ai

Java is required in this project because it lays the foundation of the writing my web app code. It's like needing to know a language in order to speak it. Maven also needs Java in order to work!



Create the Application

What I did in this step

In this step, I will generate a web app inside my EC2 instance, using Maven and Java.

Installing Remote - SSH

I installed Remote - SSH, which is an extension within VS Code. This extension lets us connect VS Code directly to a remote server, like an EC2 instance.

SSH configuration details

Configuration details required to set up a remote connection include the host (i.e. like EC2 instance's address), the identity file (i.e. the location of our private key) and the user (i.e. the user that I'm logging into for my instance).



Create the Application

Exploring the project structure

Using VS Code's file explorer, I could see a bunch of folders and subfolders that define my web app. Folders can be expanded by clicking on all of the angled brackets.

These folders organise different parts of the web app; for example, the resources sub-folder store connection details, while the webapp subfolder store web app files for the look and feel the web app. src is the parent folder that contains all the source code for the web app.

```

✓ NEXTWORK-WEB-PROJECT [SSH: EC2-3-14-65-242.US..
  ✓ src/main
    ✓ resources
    ✓ webapp
    ✓ WEB-INF
      web.xml
      index.jsp
      pom.xml
```



Using Remote - SSH

What I did in this step

In this step, I will connecting VS Code with my EC2 instance. This is different from just connecting the terminal (i.e. our local computer) to our instance.

Updating the web app

The index.jsp is the file in our web app that defines both HTML content (i.e. the static elements that go into our web app's page), as well as any code for generating dynamic content (i.e. content that's always changing).

I edited index.jsp by updating the HTML code to also say Hello {YOUR NAME} I'm also added a paragraphe i.e. some text that says "this is my web app working!"



index.jsp X

src > main > webapp > index.jsp > html > body

```
1  <html>
2  <body>
3
4  <h2>Hello World It's Hayder</h2>
5
6  <p>This is my NextWork web application working!</p>
7  |
8  </body>
9  </html>
10
```



Using nano

Additional improvements

In this secret mission, I will editing code WITHOUT an IDE! This is going to teach us why IDEs like VS Code are so well popular, and how they make editing code mush easier.

Terminal vs IDE

An alternative to using IDEs is within the terminal itself. To edit index.jsp, I ran 'cd' commands to navigate into the terminal to the right subfolder, and then ran the 'nan-o' command to use nano to open up and start editing the index.jsp file.

Compared to using an IDE, editing index.jsp in the terminal felt a littel less intuitive to navigate your cursor. I'd be more likely to use an IDE if i had lots of bits of code to update and delete, or copy and paste (i.e. lots of nevigation within the file)

Verifying my work

To verify my editing work in the terminal, I opened back up our VS Code window that had the SSH connection to our EC2 instance. It was possible to see my changes in VS Code right away because the window shared the same SSH connection to our instance as our local terminal.



Hayder Ben Abdelhamid

NextWork Student

nextwork.ai

```
GNU nano 8.3 index.jsp
<html>
<body>

<h2>Hello World It's Hayder</h2>

<p>This is my Nextwork web application working!</p>

</body>
</html>
```

[Read 9 lines]

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	^U Undo	^M Set Mark	^_ To Bracket
^X Exit	^R Read File	^N Replace	^V Paste	^J Justify	^/_ Go To Line	^R Redo	^G Copy	^B Where Was

